Group(B) Assignment No. 1

Title - Write a code in Java for a simple Word count appln that counts the number of occurrences of each word in a given input set using the Hadoop MapReduce framework on local-standalone set-up.

prerequisite :-

1. Java Installation - Java (open Jdk 11)
   java version
2. Hadoop Installation - Hadoop 2 or higher

Theory:-

- Hadoop mapReduce is a software frame-work for easily writting appln which process vast amounts of data (multi-terabytes) data-sets in parallel on large clusters (thousands of nodes) of commodity hardware in a reliable fault-tolerant manner
- A manner MapReduce job usually splits the input data-set into independent chunks which are processed by the map tasks in a completely parallel manner. The framework sorts the outputs of the maps. which are then input to the reduce tasks.

Page No
Date

Word count Example-
Wordcount example reads text files & counts how often words occur. The input is text files & the output is text files, each line of which contain a word & the count of how often it occured, seprated by a tab.
As an optimization, the reducer is also used as combiner on the map outputs. This reduces the amount of data sent across the network by combining each word into a single recor

steps to execute:-
1> create a text file your local machine and write some text into it.
$ nano. data. txt

2> In this ex, we find out the frequency of each word exists in this text file.
3> create a directory in HDFS, where to kept text file.
$ hdfs dfs-mkdir /test
4. Upload the data.txt file on HDFS in the specific directory.
$hdfs dfs- put /home/ category /data.txt /
test

5. create jar file of this program & name it ~~ou~~ count ~~el~~ word demo.jar

6. Run the jar file
hadoop jar/ home/ codegyani / wordcount demo.jar com.javapoint. wc - Runner/ test/data-txt /r-output

The output is stored in /r-output/ part-00000

7. Now execute the command to see the output.
hdfs dfs - cat /r-output/part-00000

conclusion:-
The practical was successful in achieving its aim.

Page No. 1

Date

Dr. D. Y. Patil
Varale Campus

Group B - Assignment-02

Title :- Design a distributed appln using Map Reduce which process a log file of a system.

Objective :-
Students should be able to process a log file of a system using the Hadoop Map-Reduce framework.

Prerequisite :-
1. Basic of Java programming

Hardware/ software Requirement
Operating System
Java
Hadoop framework
Development tools - Eclipse / VS code

Theory :-
Map Reduce is a programming model developed by Google for processing and generating large datasets.

Map phase :-
MapReduce is a programming model developed by Google for processing and generating large datasets.

Page No.
Date

Reduce phase-
The Reduce funⁿ groups all the intermediate values by keys and performs computation, such as counting or aggregation, to produ[ce] the final output.

System Log file Analysis using Map Reduce:-
In this practical, the goal is to design a distributed appln that processes a log file to count occurr-ences of specific log messages, such as ERROR, WARNING and INFO.

Steps to Design a Distributed Appln using Map Reduce to process a Log file :-
Step 1- Understand the problem
Step 2 - Set up the Hadoop Environm[ent]
Step 3- Store the Log file in HDFS
Step 4 - Write the Mapper class
<Log-Message, 1>

Step 5- Write the Reducer class
<Log Message, Total-count>

Step 6- Create a driver program

step 7- Execute the MapReduce job.
step 8- view the output

hdfs dfs -cat /output/part-r-0000

Step 9- Analyze the result
step 10- conclusion

Conclusion:-
The distibuted appln using MapReduce efficiently processes large log files by parallelizing tasks across multiple nodes.

Dr.D.Y.PATIL
Varale Campus

Group B - Assignment - 03

Title - Write a simple program in SCALA using Apache spark framework.

Objective :-
Understand how to create simple Apache simple Apache spark insta-llation appln using SCALA.

prerequisites :-
1. Apache spark Installation
2. Scale Installation
3. Java Development kit (JDK)
4. SBT (scala Build Tool)

Hardware / software Requirements :-
- operating system
- Hard disk
- Apache spark
- SCALA
- JAVA (JDK)
- Text Editor / IDE

Theory :-
Apache spark is an open-source distributed computing system designed for fast and flexible processing of large scale data.

Spark improves upon the limitations of Hadoop MapReduce by offering in memory data storage, which makes computations much faster for iterative algorithms and real-time analytics.

1. RDD (Resilient Distributed Datasets)
- The core data structure in spark.
- Immutable, fault-tolerant distributed collections of objects.

Algorithm :-

1. Start
2. Initialize the spark configuration and create a spark context.
3. Load the input ~~text~~ file using the flatMap () transformation.
4. Map each word to a key-value pair using map (), where the key is the word and the value is 1.
5. Split each line of the file into individual words using the flatmap () transformation.
6. Group and aggregate the counts for each word using the reduce ByKey () transformation.

7. collect the final word count results using the collect () action.
8. print the word-count pairs on the console.
9. Stop the sparkcontext to release the resources.
10. End

conclusion :-
In this practical, we implemented a simple word count program using SCALA and Apache spark successfully.